

🕒 Cập nhật tháng 8 năm 2024

[Bài đọc] Nguyên tắc của kiến trúc REST

Trong phát triển ứng dụng web và di động hiện đại, việc **trao đổi dữ liệu nhanh chóng, hiệu quả và dễ mở rộng** là yếu tố sống còn. REST (Representational State Transfer) đã trở thành **chuẩn mực** để xây dựng API, nhờ sự **đơn giản, hiệu quả và tính linh hoạt cao**

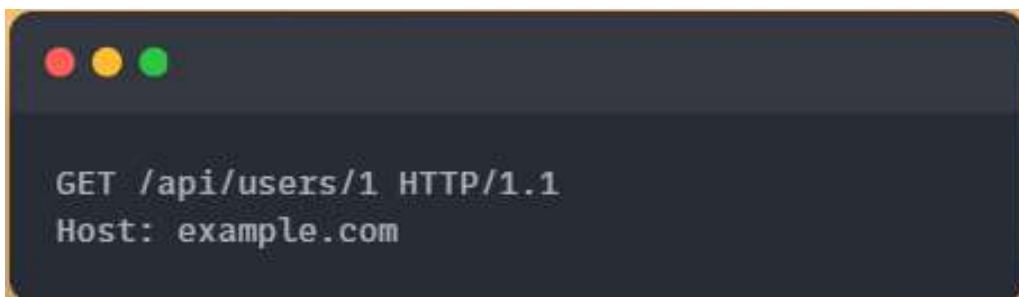
Ví dụ: Khi bạn sử dụng **Facebook**, mọi thao tác như đăng bài, xem bình luận hay tải ảnh đều dựa trên các API được thiết kế theo **REST** để client (ứng dụng trên điện thoại) có thể giao tiếp với server. Điều này giúp hàng triệu người dùng truy cập cùng lúc mà hệ thống vẫn hoạt động ổn định

1. REST là gì?

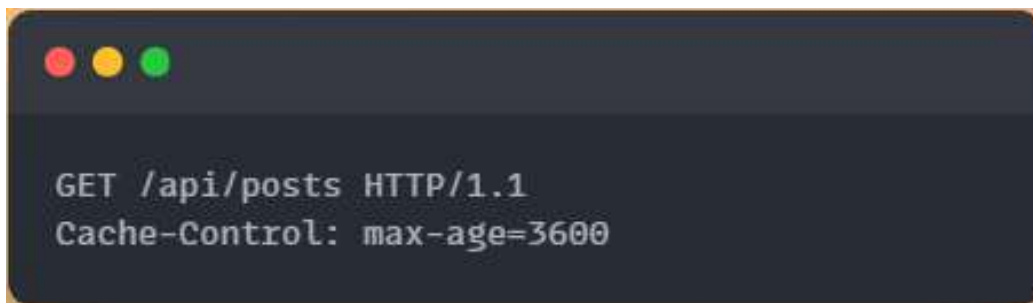
- **Định nghĩa:** REST là một **kiến trúc phần mềm phân tán**, sử dụng giao thức HTTP để trao đổi dữ liệu giữa client và server
- **Ý nghĩa:** REST giúp API trở nên **đơn giản, dễ hiểu, dễ mở rộng**, và vì vậy được sử dụng trong hầu hết các hệ thống lớn hiện nay (Google, Facebook, Twitter, GitHub...)

2. Nguyên tắc cơ bản của REST

- **Stateless (Phi trạng thái)**
 - **Khái niệm:** Mỗi request từ client đến server là **độc lập**, server không lưu trạng thái giữa các lần gọi
 - **Ý nghĩa:** Giúp hệ thống **dễ mở rộng**, giảm tải việc quản lý session phức tạp

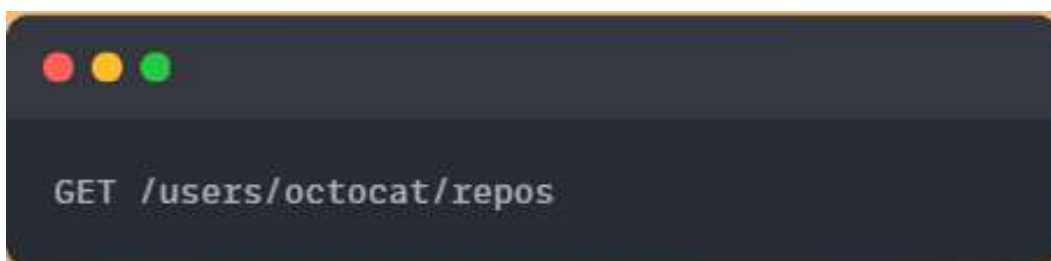


- **Giải thích:** Server chỉ dựa vào request hiện tại để trả về thông tin, không cần nhớ lịch sử trước đó
- **Client - Server (Tách biệt client và server)**
 - **Khái niệm:** Client (giao diện) và server (xử lý logic, dữ liệu) hoạt động độc lập
 - **Ý nghĩa:** Giúp dễ dàng thay đổi và bảo trì hệ thống
 - Ví dụ:
 - Client: Ứng dụng Facebook trên điện thoại
 - Server: Máy chủ lưu trữ dữ liệu bài viết, ảnh, tin nhắn
- **Caching (Bộ nhớ đệm)**
 - **Khái niệm:** Cho phép lưu trữ tạm thời dữ liệu đã truy vấn
 - **Ý nghĩa:** Giúp giảm tải server, tăng tốc độ phản hồi



```
GET /api/posts HTTP/1.1
Cache-Control: max-age=3600
```

- **Giải thích:** Client có thể lưu dữ liệu bài viết trong 1 giờ, không cần gọi lại server
- **Uniform Interface (Giao diện đồng nhất)**
 - **Khái niệm:** Dùng các URI và phương thức HTTP chuẩn hóa để truy cập tài nguyên
 - **Ý nghĩa:** Làm API dễ hiểu, dễ dùng, dễ tích hợp
 - Ví dụ (GitHub API):



```
GET /users/octocat/repos
```

- **Giải thích:** API sử dụng **GET** để lấy danh sách repo của user **octocat**
- **Layered System (Hệ thống phân tầng)**
 - **Khái niệm:** Hệ thống REST có thể tổ chức thành nhiều tầng (client ↔ proxy ↔ server)
 - **Ý nghĩa:** Tăng bảo mật, linh hoạt và khả năng mở rộng

- Ví dụ: Cloudflare CDN làm **tầng trung gian** giúp giảm tải server gốc

- **Code on Demand (Tải mã theo yêu cầu)**

- **Khái niệm:** Cho phép server gửi mã thực thi (thường là JavaScript) xuống client
- **Ý nghĩa:** Giúp tăng khả năng xử lý của client, giảm tải cho server
- Ví dụ: Ứng dụng web tải **script** từ server để hiển thị biểu đồ ngay trên trình duyệt

3. Bảng tổng hợp nguyên tắc của REST

Nguyên tắc	Ý nghĩa chính	Ví dụ thực tế
Stateless	Mỗi request độc lập	API Twitter
Client – Server	Tách biệt xử lý và giao diện	Facebook App & Server
Caching	Lưu dữ liệu tạm, giảm tải server	Google Search
Uniform Interface	Chuẩn hóa API bằng URI + HTTP method	GitHub API
Layered System	Hệ thống phân tầng, tăng bảo mật & mở rộng	Cloudflare CDN
Code on Demand	Tải mã theo yêu cầu từ server	Web app chạy script JS động